

Práctica 1 · Mi primera app Android (Temas 1-3) · Individual

Mauro Rodriguez Ponce, 2-A

7/10/2026



Índice

Práctica 1 · Mi primera app Android (Temas 1-3) · Individual	1
Parte A · Análisis del desarrollo móvil	3
Parte B · Especificación de tu app	7
Parte C · Entorno de desarrollo	12
Parte D · Recursos de la aplicación	21



Parte A · Análisis del desarrollo móvil

1. Ventajas e inconvenientes del desarrollo para dispositivos móviles frente al desarrollo de escritorio o web. Incluye al menos 4 limitaciones concretas (batería, memoria, conectividad, tamaño de pantalla, fragmentación...) y cómo afectarían a tu app.

- Ventajas:

Una de las ventajas frente a escritorio y web, por ejemplo, es que el móvil al ser portátil siempre está a mano, así que se puede apuntar una película en cualquier momento.

También se pueden usar las notificaciones ("Hoy toca peli: tienes 5 pendientes") y compartir títulos con otras apps. También funciona sin conexión si se guardan los datos en el propio dispositivo.

Otra ventaja más sería que se distribuye a muchos usuarios a través de Google Play.

- Desventajas:

Las desventajas que tiene frente a las de escritorio puede ser, por ejemplo, que los recursos de los dispositivos móviles son más limitados (batería, memoria, CPU). Otra desventaja sería que hay pantallas pequeñas y táctiles, con muchos tamaños distintos.

También, otra desventaja, sería la fragmentación de versiones de Android y de fabricantes. Y para terminar habría que cumplir las políticas de la tienda y actualizar la app con cada versión nueva del sistema.



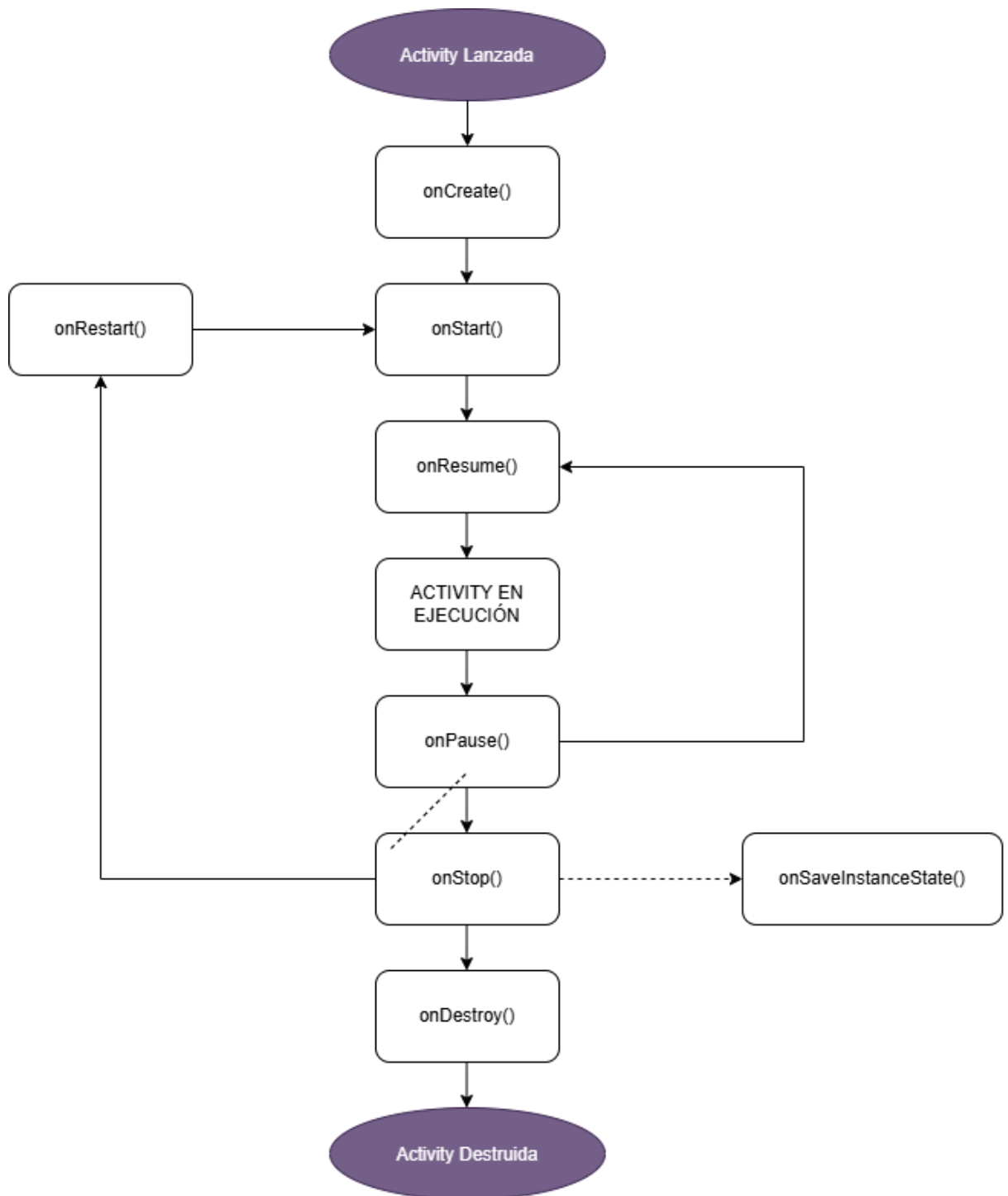
- Limitaciones Concretas:
 - **Batería:** Es una app ligera, pero un tema muy luminoso o animaciones constantes harían que se gastara más la batería.
 - **Memoria:** Si la app es muy pesada y está poco optimizada pueden llenar la RAM, y Android puede cerrar la app en segundo plano y perder un formulario a medio llenar.
 - **Tamaño de pantalla:** Al haber tantos tamaños puede si no está bien ajustada la app la aplicación no podrá ni leerse, ni verse bien en móviles pequeños y en tablets.
 - **Fragmentación:** Hay muchos modelos y versiones de Android.

2. Elección tecnológica: tabla comparativa nativo / híbrido / multiplataforma (lenguajes, rendimiento, acceso al hardware, coste, un ejemplo de framework de cada tipo). Justifica por qué tu app se desarrollará en nativo Android.

	Nativo	Híbrido	Multiplataforma
Lenguajes	Java / Kotlin (Android), Swift (iOS)	HTML, CSS y JavaScript dentro de un WebView	Dart, JavaScript/TypeScript, C#
Rendimiento	Máximo	Menor, depende del WebView	Alto, cercano al nativo
Acceso al hardware	Total y directo	Limitado, mediante plugins	Amplio, a veces con plugins
Coste	Alto si se quiere iOS y Android (dos bases de código)	Bajo (una base web reutilizable)	Medio (una base para ambas plataformas)
Framework de ejemplo	Android SDK	Ionic + Capacitor	Flutter / React Native



3. Ciclo de vida de una Activity: dibuja el diagrama y describe qué ocurriría en tu app en cada uno de estos casos: el usuario recibe una llamada, gira el móvil y pulsa el botón Atrás.



- Qué ocurre:



-El usuario recibe una llamada:

La Activity pasa a onPause(). Si la llamada ocupa toda la pantalla, también pasa a onStop(). En onPause() guarda el borrador. Al colgar se ejecuta onRestart() → onStart() → onResume() y así el usuario continúa con la aplicación

-El usuario gira el móvil:

Android destruye y recrea la Activity: onPause() → onStop() → onSaveInstanceState() → onDestroy() → onCreate() → onStart() → onResume(). Los EditText con id conservan su texto, pero otros valores (la nota elegida, el filtro "vistas/pendientes") se perderían, así que los guardó en onSaveInstanceState() y los restauró en onCreate(). Además puedo ofrecer un layout-land con la lista y la ficha en paralelo.

-El usuario pulsa Atrás:

En la pantalla principal la Activity termina: onPause() → onStop() → onDestroy(). En la pantalla "Añadir película" se vuelve a la lista y se descarta lo no guardado, por lo que muestro un diálogo para no perder datos sin querer.



Parte B · Especificación de tu app

1. Idea y público objetivo (5-8 líneas) y 3 objetivos de la app redactados de forma medible.

PelisPendientes es una app Android para apuntar las películas y series que quiero ver y no perderlas ni olvidarme de ellas. Así, cuando alguien me recomiende una peli o serie, la añado en segundos indicando el nombre, si es película o serie y dónde está disponible. Cuando ya haya visto la peli/serie, la marco como "vista" y le pongo una nota de 1 a 5. Todo se guarda en el propio móvil, así que funciona sin conexión.

Público objetivo: jóvenes y adultos que usan varias plataformas de streaming y acumulan recomendaciones que luego olvidan.

Nº	Objetivo	Cómo se mide
Objetivo 1	Añadir un título de manera rápida.	Cronometrando 5 intentos en el emulador.
Objetivo 2	Mostrar una lista de 200 títulos en menos de 1 s sin bloqueos.	Probando con 200 registros de prueba.
Objetivo 3	Que 5 de 5 personas de prueba completen "añadir, marcar como vista y puntuar" sin ayuda en menos de 1 minuto.	Prueba con compañeros, anotando tiempos.



2. **Requisitos funcionales:** mínimo 5, numerados (RF-01...), con prioridad (alta/media/baja).

ID	Requisito	Prioridad
RF-01	La app permite añadir un título indicando nombre, tipo (película/serie) y plataforma.	Alta
RF-02	La app muestra una lista con todos los títulos guardados.	Alta
RF-03	El usuario puede marcar un título como "visto".	Alta
RF-04	Al marcar un título como visto, el usuario le asigna una nota de 1 a 5.	Alta
RF-05	El usuario puede filtrar la lista por "pendientes", "vistas" o "todas".	Media

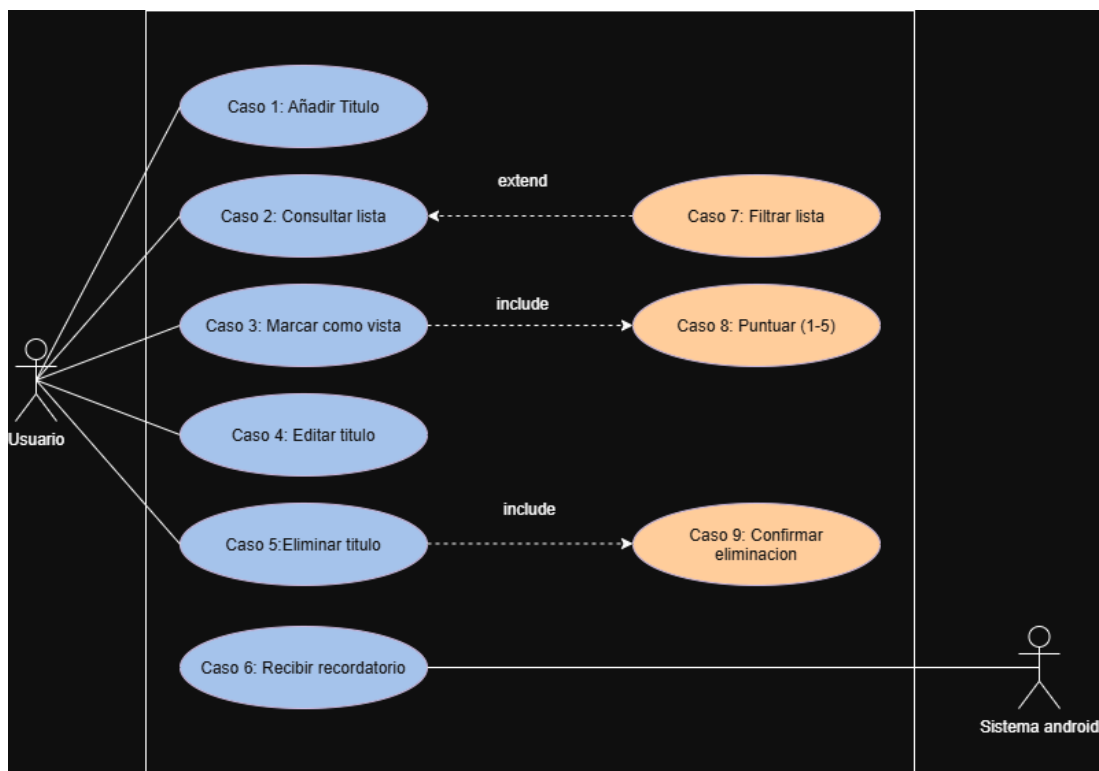
3. **Requisitos no funcionales:** mínimo 4 (RNF-01...), verificables. Ejemplo: "la pantalla principal carga en menos de 2 s", no "la app es rápida". Al menos uno de usabilidad, uno de compatibilidad (versión mínima de Android y por qué) y uno de rendimiento.

ID	Tipo	Requisito verificable
RNF-01	Usabilidad	Las acciones principales (añadir, marcar como visto, puntuar) se hacen en un máximo de 3 toques. Todos los elementos táctiles miden al menos 48 dp y el contraste texto/fondo es de al menos 4,5:1.
RNF-02	Compatibilidad	API mínima 24 (Android 7.0), porque cubre la gran mayoría de móviles en uso y permite usar imágenes vectoriales, que desde la API 21 se soportan de forma nativa. Se comprueba ejecutándola en un AVD con API 24.
RNF-03	Rendimiento	La pantalla principal es visible en menos de 2 s tras abrir la app (arranque en frío en emulador), y el APK de depuración pesa menos



		de 10 MB.
RNF-04	Fiabilidad	Tras 10 giros de pantalla consecutivos no se pierde ningún dato introducido, ni el filtro seleccionado.

4. Casos de uso: diagrama con actores y al menos 4 casos, más la ficha detallada de 2 de ellos (actor, precondition, flujo principal, flujo alternativo, postcondición).





- **Fichas detalladas:**

- Ficha Caso 1: Añadir título

Campo	Contenido
Actor	Usuario
Precondición	La app está abierta y muestra la pantalla principal.
Flujo principal	1. El usuario pulsa el botón "Añadir". 2. La app muestra el formulario con nombre, tipo y plataforma. 3. El usuario escribe el nombre, elige el tipo y la plataforma. 4. El usuario pulsa "Guardar". 5. La app valida los datos y guarda el título como "pendiente". 6. La app vuelve a la lista y muestra el nuevo título.
Flujo alternativo	Alternativa 1 (pasos 2 a 4): llamada o giro de pantalla. Entra una llamada o se gira el móvil con el formulario abierto. La app conserva lo escrito y lo recupera al volver (RF-08). El flujo continúa en el paso donde estaba. Alternativa 2 (paso 3): el usuario pulsa Atrás. La app pregunta "¿Descartar cambios?". Si confirma, vuelve a la lista sin guardar. Si cancela, sigue en el paso 3. Alternativa 3 (paso 4): nombre vacío. La app marca el campo con un mensaje de error y no guarda. El flujo vuelve al paso 3. Alternativa 4 (paso 4): título duplicado. La app avisa de que ya existe y pregunta si se quiere añadir igualmente. Si acepta, continúa en el paso 5. Si no, vuelve al paso 3.
Postcondición	El título queda guardado en el dispositivo con estado "pendiente" y aparece en la lista.



- Ficha Caso 3: Marcar como vista y puntuar

Campo	Contenido
Actor	Usuario
Descripción	El usuario indica que ya ha visto un título y le asigna una nota.
Precondición	Existe al menos un título con estado "pendiente" en la lista.
Flujo principal	1. El usuario toca un título pendiente de la lista. 2. La app muestra la ficha del título. 3. El usuario pulsa "Marcar como visto". 4. La app muestra el selector de nota de 1 a 5 (CU-08, include). 5. El usuario elige una nota y confirma. 6. La app guarda la nota y cambia el estado a "visto". 7. La app vuelve a la lista, donde el título aparece como visto con su nota.
Flujo alternativo	Alternativa 1 (paso 1): título ya visto. El título seleccionado ya estaba marcado como visto. La app ofrece "Cambiar nota" en lugar de "Marcar como visto" y el flujo continúa en el paso 4. Alternativa 2 (paso 5): el usuario cancela el selector. El título sigue pendiente y no se guarda ninguna nota. La app vuelve a la ficha del título (paso 2). Alternativa 3 (paso 5): confirmar sin nota. El usuario pulsa confirmar sin elegir un valor. La app le pide elegir una nota entre 1 y 5 y el flujo vuelve al paso 5.

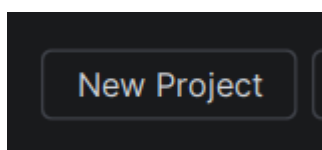


Postcondición	El título queda con estado "visto" y una nota entre 1 y 5, y deja de aparecer en el filtro "pendientes".
----------------------	--

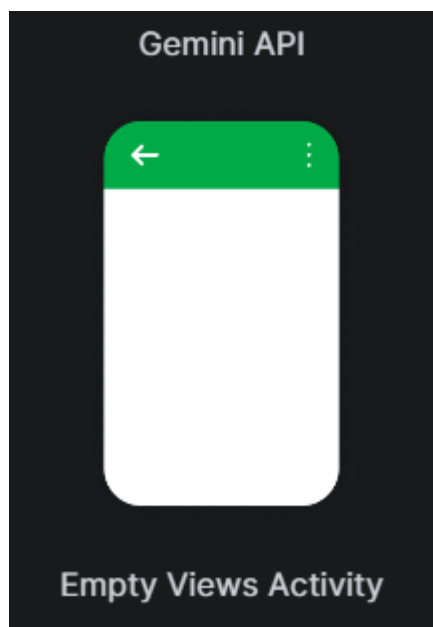
Parte C · Entorno de desarrollo

1. Crea el proyecto en Android Studio con el nombre **PMDM_P1_TuNombreApellido** y un paquete propio (**es.medac.tunombre.app**).

Primero haremos click en **"New project"**.



Después se nos abrirá una ventana. En esta ventana seleccionaremos **"Empty view activity"**.





Lo siguiente será ponerle un nombre a nuestro proyecto y un paquete propio. También nos aseguraremos que este en Java.

Name: PMDM_P1_MauroRodriguezPonce

Package name: es.medac.maurorodriguezponce.app

Save location: C:\Users\mauro\AndroidStudioProjects\PMDM_P1_MauroRodriguezPonce

Language: Java

Minimum SDK: API 24 ("Nougat"; Android 7.0)

Your app will run on approximately 99,2% of devices.
[Help me choose](#)

Build configuration language: Kotlin DSL (build.gradle.kts) [Recommended]

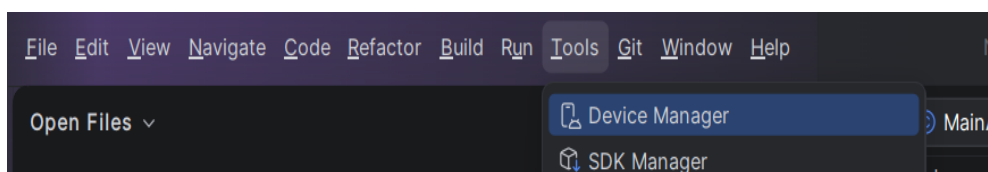
Una vez creado veremos esto:

```
1 package es.medac.maurorodriguezponce.app;
2
3 import ...
4
5
6
7
8
9
10
11 public class MainActivity extends AppCompatActivity {
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         EdgeToEdge.enable(this);
17         setContentView(R.layout.activity_main);
18         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) {v, insets} -> {
19             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
20             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
21             return insets;
22         });
23     }
24 }
```

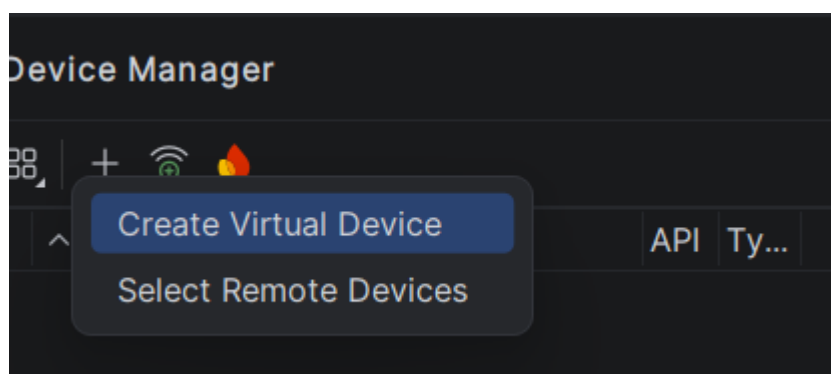


2. Configura dos dispositivos virtuales (AVD) distintos: un móvil y una tablet (u otro móvil con distinta densidad). Indica en una tabla su resolución, densidad (dpi) y versión de API.

Para configurarlos lo primero será seleccionar Tools y hacer click en Device Manager.



Al seleccionar Device Manager nos aparecerá una ventana donde podremos agregar un dispositivo pulsando +. Le daremos a



Cuando le demos nos aparecerá una ventana donde tendremos que elegir nuestro dispositivo. En mi caso elegiré un Pixel 7.

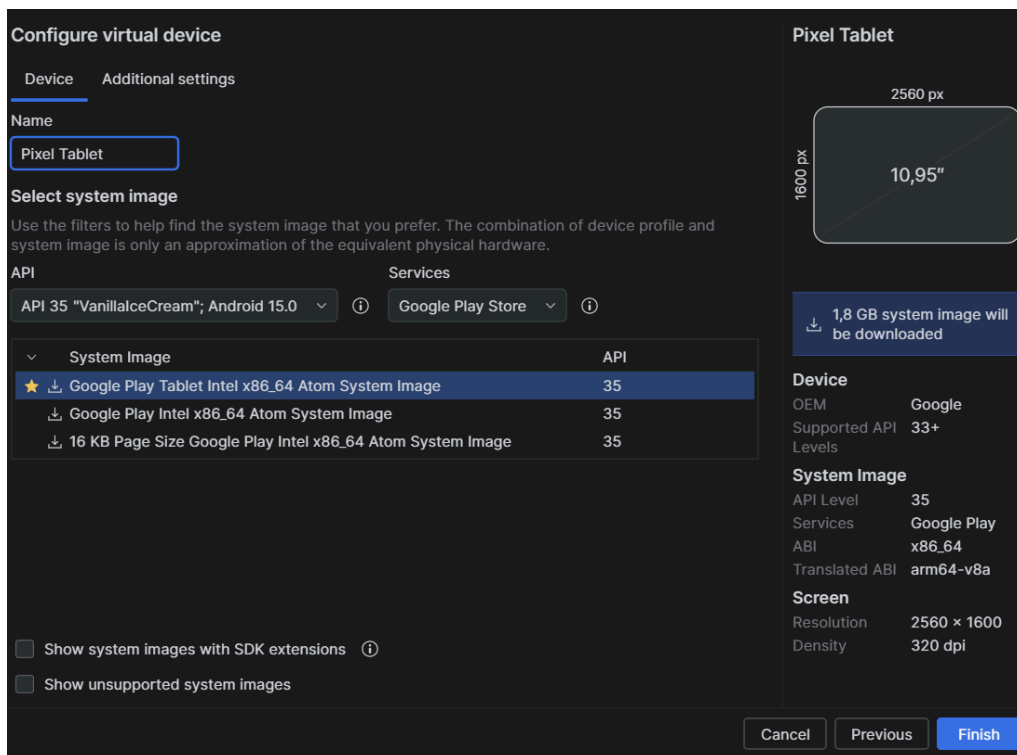
Pixel 7 Pro	33+	1440	3120	300 dpi
Pixel 7	33+	1080	2400	420 dpi

También añadir una tablet.

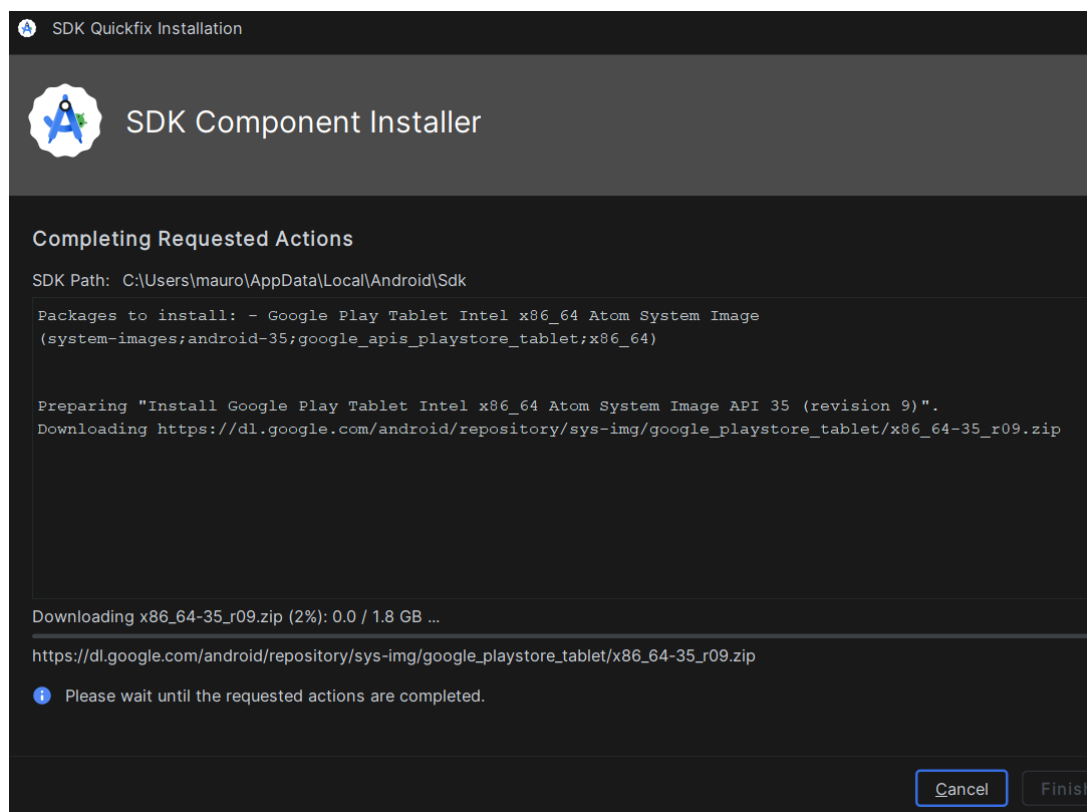
Pixel Tablet	33+	2560	1600	320 dpi
--------------	-----	------	------	---------



Al añadir un dispositivo nos saldrá la siguiente ventana:



Al darle a finish, tendremos que esperar a que se descargue.





Cuando termine de descargarse ya se habrá añadido la tablet.

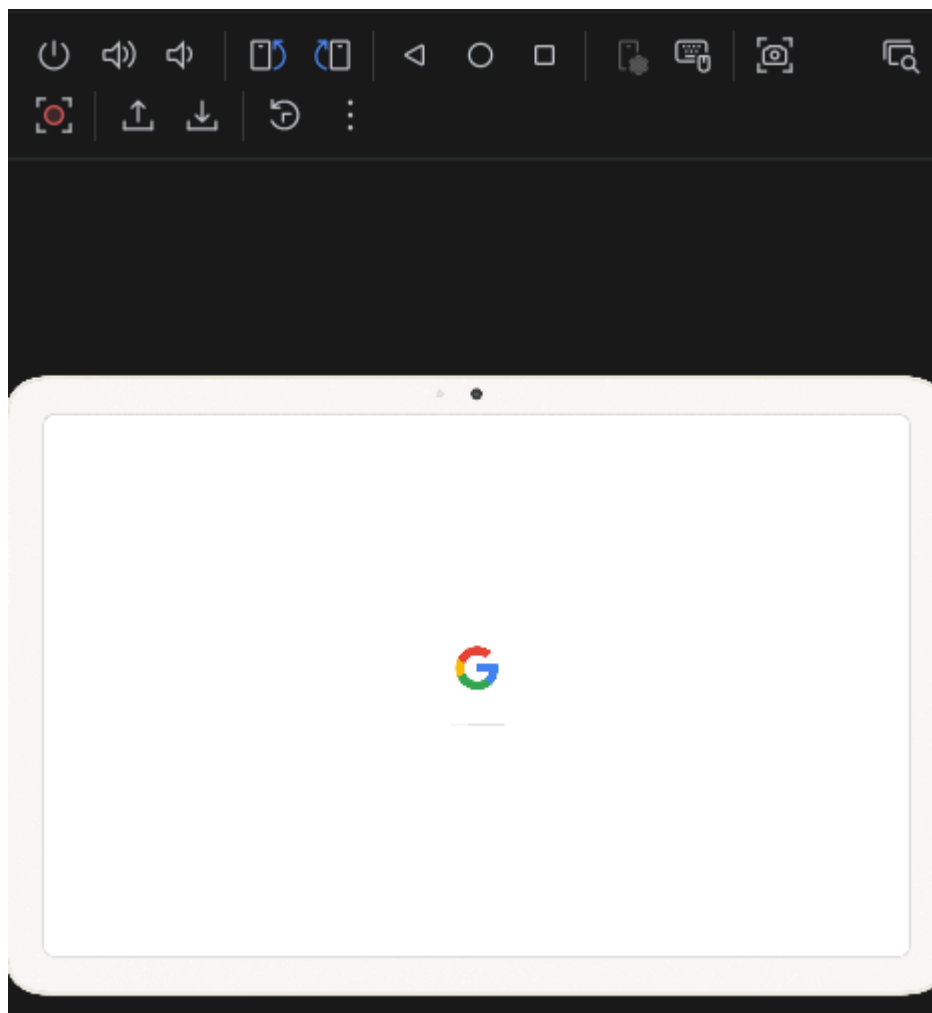
Los dispositivos nos quedarán así:

-Movil:





-Tablet:



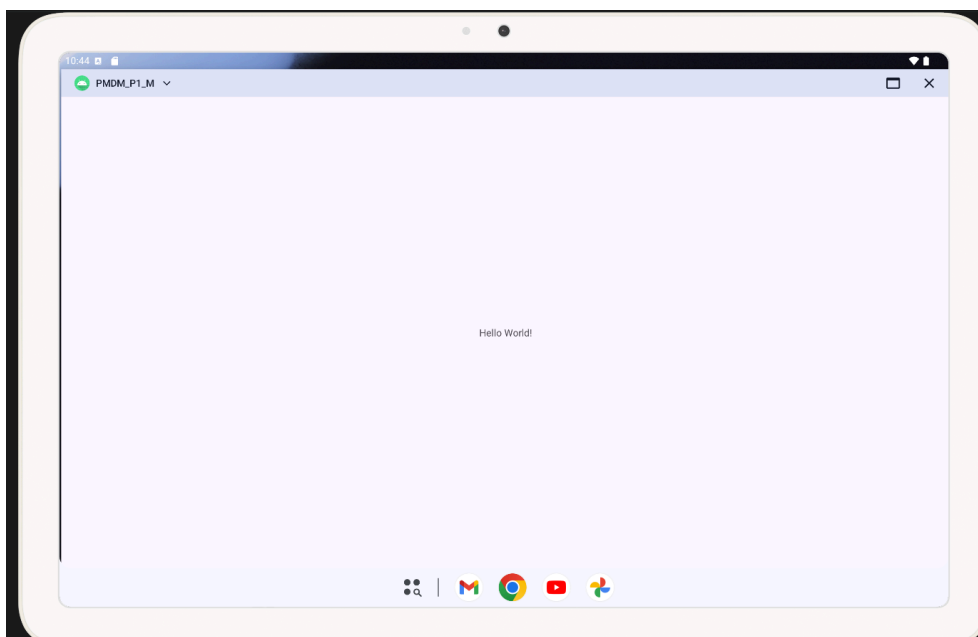
TABLA

	AVD 1	AVD 2
Nombre	Movil_Pixel8_API34	Tablet_PixelTablet_API34
Tipo	Móvil	Tablet
Resolución (px)	1080 x 2400	2560 x 1600
Densidad (dpi)	420dpi	320dpi
Versión de API	37.2	35

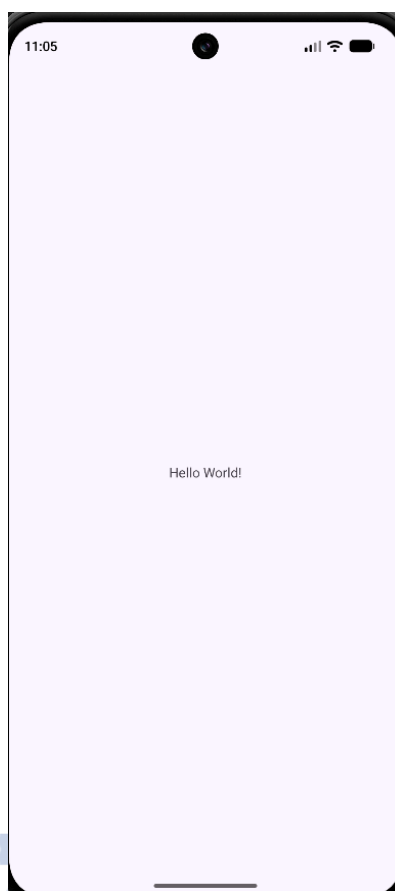


3. Ejecuta la app en ambos y aporta capturas. Opcional: ejecución en un móvil físico o en NOX.

Captura de la aplicación en tablet:



Captura de la aplicación en móvil:





4. Explica brevemente la estructura del proyecto: **AndroidManifest.xml**, **java/**, **res/** y los ficheros **Gradle**, y la relación entre la **Activity (Java)** y su **layout (XML)**.

AndroidManifest.xml (app/src/main/): es el documento donde se declara la app al sistema. Contiene el paquete, el nombre (android:label), el icono (android:icon), el tema (android:theme), las Activities de la app y los permisos que se solicitan. La Activity principal lleva un intent-filter con MAIN y LAUNCHER, que indica a Android que es la que debe abrirse al pulsar el icono.

java/ (app/src/main/java/es/medac/.../): contiene el código Java. Aquí está MainActivity.java, que hereda de AppCompatActivity y define el comportamiento de la pantalla. También hay dos carpetas marcadas como (androidTest) y (test), destinadas a pruebas.

res/ (app/src/main/res/): son los recursos de la app, separados del código:

layout/: interfaces en XML, como activity_main.xml.

values/: textos (strings.xml), colores (colors.xml), medidas (dimens.xml) y temas (themes.xml).

drawable/: imágenes y formas, vectoriales o de mapa de bits.

mipmap-*/: iconos del lanzador en distintas densidades.

Ficheros Gradle: son el sistema de compilación.

settings.gradle(.kts): define el nombre del proyecto y los módulos incluidos (aquí, :app).

build.gradle(.kts) de la raíz: configuración común a todo el proyecto (plugins).



app/build.gradle(.kts): configuración del módulo app: namespace, applicationId, minSdk (24), targetSdk, compileSdk, versión y dependencias (por ejemplo AppCompatActivity, Material y ConstraintLayout).

gradle.properties y gradle/libs.versions.toml/wrapper: parámetros de Gradle y versiones de las librerías.

Relación Activity (Java) ↔ layout (XML): el layout describe qué se ve (botones, textos, imágenes) y la Activity decide qué hace. Se conectan en onCreate() con setContentView(R.layout.activity_main). Android genera la clase R, que da un identificador numérico a cada recurso, y así el código accede a él (R.layout.activity_main, R.string.app_name, R.id.boton). Para manejar una vista concreta desde Java se usa findViewById(R.id.x) o View Binding.



Parte D · Recursos de la aplicación

Imagenes App en dispositivos

